Build the Radar

The mock comes out.
Real listings go in.

Today your agent stops inventing data and goes out to get it. First source needs no API key and no signup. But calling an API isn't the skill — the skill is what your system does when that API is down.

RECAP · WHERE YOU SHOULD BE

Where we left off

1. A repo with a **steering file** — the rules Kiro follows in every interaction.
2. A **config.yaml** with your real role, city and skills.
3. A **storage module** — the only thing in the project allowed to touch the database.
4. An **Orchestrator** that reads state, decides, and delegates to agents from a registry.

> THE NUMBER THAT HAS TO BE WORKING
>
> Run the cycle twice. Second run says **4 new**, not 12. If it doesn't — pause the video and fix that first.

WHAT JUST CHANGED ABOUT YOUR PROJECT

Your code now talks to strangers.

Until today every line you had was deterministic — same input, same output, forever. From today your project depends on a machine you don't own, run by people you've never met, who owe you nothing. They can change the response shape, rate-limit you, or be down at 6 AM.

AND THE WORST ONE

A **200 OK** with an empty array. Nothing throws, nothing logs, nothing looks broken — and your database quietly stops growing.

FAILURE MODES · ALL FOUR HAVE BORING ANSWERS

Four ways your Fetcher dies

1. **The timeout that never times out.** The connection hangs and your code waits forever.

   Fine on your laptop — you hit Ctrl-C. On a 6 AM scheduled runner the platform kills the job and bills you. Every call gets a timeout today.

2. **The shape change.** company_name became company. Nothing crashes.

   You just write None into every row. Blank companies, strange scores, and no error anywhere in the log.

3. **The cascade.** Second source goes down and takes the whole cycle with it.

   Zero listings because one of two boards had a bad day. THIS is the one that will actually happen to you.

4. **The leaked key.** Week 4, you push to deploy, and your token goes public.

   Bots scan for exactly this, continuously. We set up the pattern before you have a key to leak — afterwards it's in the history forever.

What you built today

1. A **source layer** where every job board is a plug-in, added by decorating a class.
2. One **HTTP helper** with a timeout and a retry — used by everything, forever.
3. A Fetcher that **survives any one source dying** and logs which one it was.
4. **Real listings** in your own database, filtered for the job you actually want.
5. A **secrets pattern** that stops you leaking a token in week 4.

The mock is still on disk, behind **use_mock_fetcher** — it's a useful diagnostic, not dead code.

WEEK 1 · ASSIGNMENT

Ship “The Job Radar”

Due **Sunday**. Tuesday's half plus today's half — this session completes it.

Tuesday — done

1. Repo initialised & pushed
2. Steering file in place
3. Loop skeleton runs, dedup visible
4. config.yaml — **your** role, city, skills

Today — the other half

1. Real Fetcher live, mock behind the flag
2. **50+ genuine live listings** — I check
3. One source failing doesn't kill the cycle
4. .env gitignored, .env.example committed

THE BAR

Fifty genuine live listings

50+

Real ones. I check, and test data is obvious — it has round numbers, no punctuation in company names, and every posted date is today.

Also due Sunday

1. run_cycle.py twice → dedup holds on **real** data
2. At least one source working; two if you did Apify
3. Mock retired behind **use_mock_fetcher: false**
4. .env gitignored — check with **git status**
5. 2–3 minute walkthrough video

THE TEN SECONDS THAT MATTER

Break a source on purpose. Film it.

Point one source at an invalid URL, run the cycle, and record what your system does about it. **This is steering rule 12 paying out.**

WHAT THE GRADER WANTS TO SEE

```
$ python run_cycle.py
orchestrator  woke · state: last fetch 26h ago
WARN  apify: FAILED (connection error) — continuing
fetcher       arbeitnow: 47 rows (12 new) | apify: FAILED
storage       12 listings written
cycle complete ✓
```

Everyone films the happy path. Almost nobody films the failure — which is exactly why it's the clearest signal that you **understand what you built** rather than having watched it get built.

THE SUBMISSION

Two to three minutes of video

1. **The cycle running** — per-source counts legible on screen.
2. **The database growing** — your real count, not a screenshot of mine.
3. **Dedup on the second run** — the number drops, and you say why.
4. **The deliberate failure** — one source broken, cycle survives.

Submit: **repo link + video** → **[submission form]** · Help: **[#your-channel]** — ask there, not in DMs.

UNLISTED YOUTUBE OR LOOM IS FINE

BADGE UNLOCKED

The Tracker

For the people who stopped refreshing job boards manually and built something that reports the market to them instead — every day, whether they're watching or not.

Three left this month: **The Decoder** · **The Automator** · **The Edge**. All four plus a verified public ship → certificate.

DO THIS IN THE NEXT HOUR

Two things while it's fresh

1. Run the cycle a few times **over the evening.**

   Let real listings accumulate. Fifty is the bar and you want margin. It costs you nothing but time you weren't spending anyway.

2. Post it on LinkedIn with **#MyEdge.**

   Screenshot the cycle output with per-source counts, or run two showing dedup. Say what it does and what surprised you.

> BETTER POST
>
> “My agent survived a dead job board this morning” beats a green checkmark — and it's more honest about what engineering actually is.

NEXT WEEK · WEEK 2 · THE DECODER

Your database has real data and no opinion about any of it.

Trigger → Orchestrator → Fetcher ✓ **Scorer ← next** **Gap Analyzer ← next**

Every listing gets a fit score against your profile and a written reason. Then the Gap Analyzer reads across all of them and tells you which single skill is costing you the most opportunities.

That's the panel nobody expects, and it's the one that changes what you do on a **Saturday.**

THE PROBLEM YOU'LL MEET NEXT WEEK

And they will lie to you.

An LLM scoring your listings will be agreeable. It'll hand you eighty-eights and ninety-twos across the board and sound confident about every one of them.

WHICH IS WHY WEEK 3 IS THE VERIFIER

A ranking where everything ranks first **is not a ranking.** But first we have to build something worth verifying.

It isn't a tutorial project anymore.

On Tuesday your project invented its own data. Today it went out and got real data, off the actual internet, filtered for the actual job you actually want — and it kept working when the network didn't.

That's a system with a memory and a failure mode it survives on purpose.

Fifty listings by Sunday. Break something on camera. See you next week.